

Assignment - 2

Q.1 What is the need for code migration? Explain the role of process to resource and resource to machine binding in code migration.

⇒

- Code migration refers to moving code from one machine to another in distributed system.
- This improves efficiency, resource utilization and performance.
- Need for Code Migration :-

1) Load balancing

⇒ Distribute workloads across multiple machines to prevent bottlenecks.

2) Performance Optimization

⇒ Moves execution closer to data sources to reduce network latency.

3) Fault tolerance

⇒ Allows processes to migrate to another machine in case of failures.

4) Resource Utilization

⇒ Allocates resources efficiently by moving code to machines with high processing power.

5) Scalability

⇒ Enables dynamic scaling of applications across multiple machines.

1) Process-to-Resource Binding

⇒ This refers to how a process is associated with resources. Three types of process-to-resource bindings exist.

- Binding by Identifier: A process is linked to a specific resource.
- Binding by Value: The process can access a copy of a resource.
- Binding by Type: The process can use any available resource that meets a requirement.

2) Resource-to-Machine Binding

⇒ This defines how a resource is linked to a physical machine.

- Fixed Binding: The resource is physically attached to a machine.
- Movable Binding: The resource can be migrated between machines.
- Unbound Resource: The resource is independent of machines.

Q.2 List desirable features of distributed file system. How are modifications propagated in file caching schemes?

⇒

- A distributed file system (DFS) allows multiple users to access and share files stored across multiple servers in a distributed networks.

- Key Desirable Features :-

1) Transparency

⇒ Users can access files without knowing where they are stored.

- The system automatically finds files, regardless of physical storage location.

- Multiple users can access file simultaneously.

2) Scalability

⇒ Supports large numbers of users and files.

- Handles increased load by distributing storage and computations.

3) Fault tolerance & Reliability

⇒ Redundancy & Replication ensure data availability in case of server failures.

- Automatic recovery mechanisms help maintain file integrity.

4) Performance

⇒ Local caching improves read/write speeds.
Load balancing distributes files requests efficiently.

5) Security

- ⇒ Authentication and Authorization protect against unauthorized access.
- Encryption ensures secure data transmission.

• File Modification Propagation in Caching Schemes:

- File caching is used in DFS to reduce access times by storing copies of frequently accessed files locally.

1) Write-Through Caching

- ⇒ Changes are immediately written to both the cache and the server.
- Ensures strong consistency but may cause performance overhead.
 - Example: Bank transactions.

2) Write-Back Caching

- ⇒ Changes are first written to cache, then periodically flushed to the server.
- Example: Temporary files in web browser.

3) Write - Invalidate cache

- ⇒ When a process modifies a file, cached copies are invalidated on other nodes.
- Other processors must fetch the updated file before using it.
- Example: Distributed database.

4) Write - Update caching

- ⇒ When a file is modified, updated data is broadcasted to all cached copies.
- Ensures all copies remain consistent but generates high network load.
- Example: Collaborative editing tools.

(A)

Rodger